МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное

Бюджетное образовательное учреждение высшего образования

МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «МКиИТ»

Лабораторная работа № 3
по дисциплине «Введение в информационные технологии»
по теме: «Функции в Python и базовые алгоритмы»

Номер группы: 2604

ФИО: Гаранский Ярослав Александрович

Проверил:

Москва 2026

Оглавление

Введение……..3
Выполнение……4
Вывод……..15

Цель работы:

изучить принципы создания и использования функций в языке Python, научиться передавать данные в функции и возвращать результаты, понять области видимости переменных, освоить работу со списками и кортежами, срезами и встроенными функциями, а также получить практический опыт реализации базовых алгоритмов обработки последовательностей и применения lambda-функций.

В ходе выполнения лабораторной работы необходимо научиться разбивать программу на логически завершённые функции, использовать параметры и возвращаемые значения, обрабатывать последовательности данных и самостоятельно реализовывать простейшие алгоритмы поиска, подсчёта и агрегирования.

Выполнение

Шаг 1. Создание и вызов функций.

На рисунке 1 представлены созданные три функции для выполнения базовых операций: greet для приветствия пользователя, square для вычисления квадрата числа, max_of_two для нахождения максимального значения из двух чисел. Все три функции были успешно созданы и вызваны в процессе тестирования.

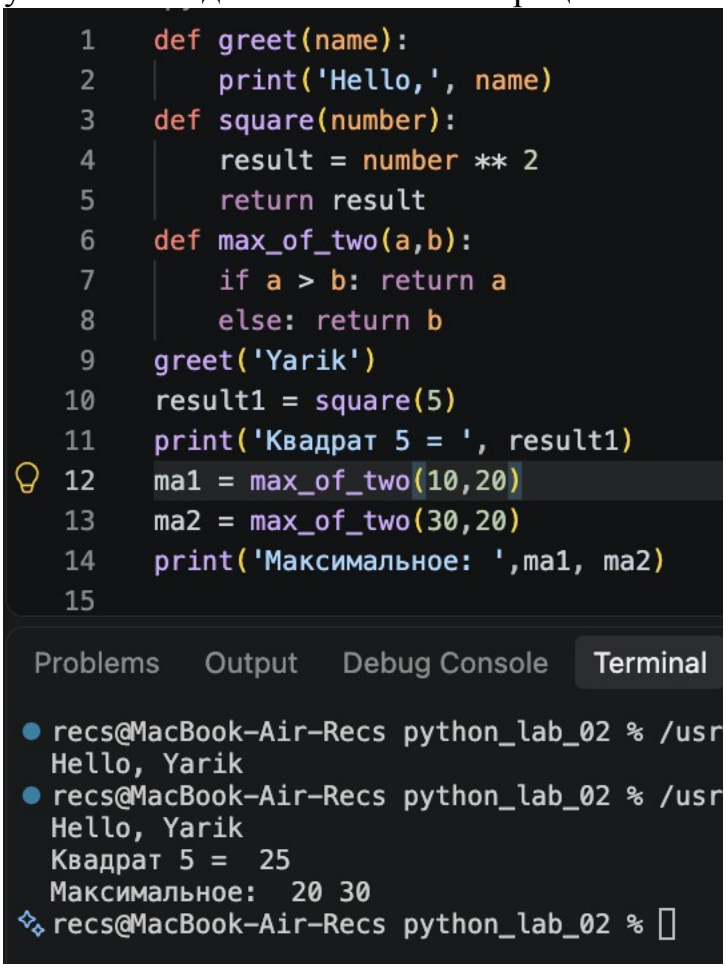


Рисунок 1 — результат работы функций greet, square, max_of_two.

Шаг 2. Аргументы и возвращаемые значения.

Созданы три функции для работы с геометрией прямоугольника — вычисления площади, периметра и вывода размеров. Все функции были успешно созданы и вызваны с различными аргументами. Вызвана функция describe_rectangle() как с аргументом unit, так и без него.
(Рис.2).

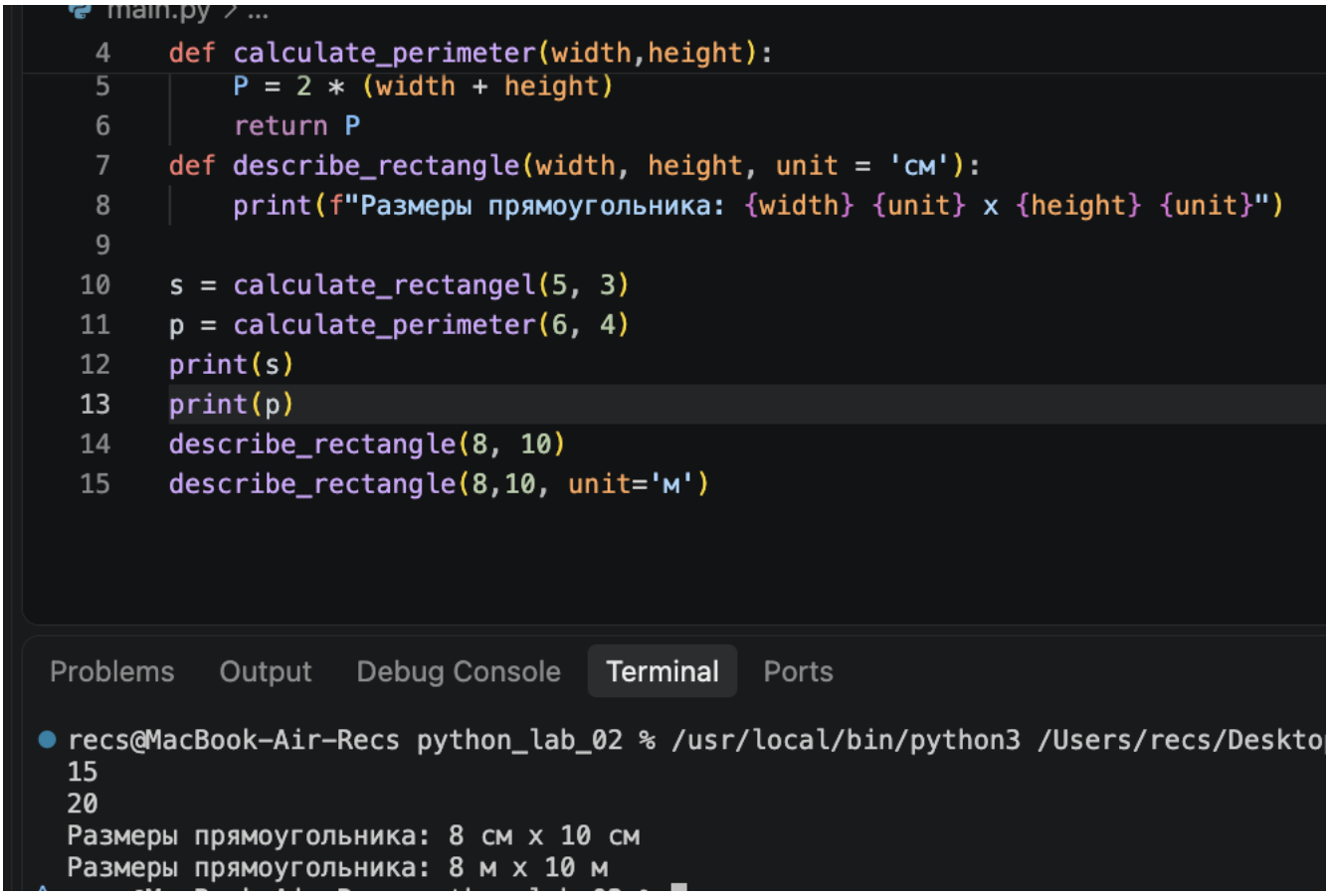


Рисунок 2 — Несколько функций.

Шаг 3. Области видимости переменных.

На рисунке 3 представлена программа, данная в методичке, и ее результат. Выводятся два разных значения потому что есть две переменные value — одна из них глобальная, а другая локальная, которая находится внутри функции. Локальная переменная не заменяет глобальную.

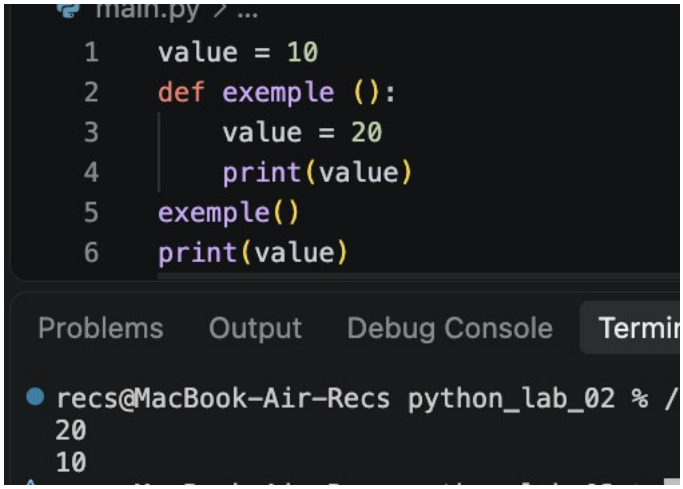


Рисунок 3 — Вывод программы.

При попытке обратиться к локальной переменной появляется ошибка так как локальная переменная существует только внутри области ее определения.(рис 4)

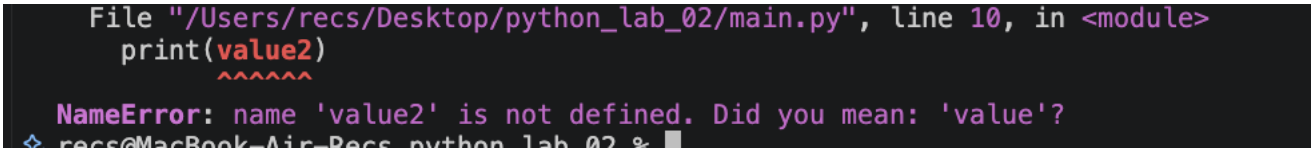


Рисунок 4 — ошибка при попытке обратиться к локальной переменной.

Шаг 4. Работа со списками.

Создан список из 10 целых чисел. Выполнены операции индексации изменение значения второго элемента, добавление нового элемента с помощью метода append(), удаление элемента с помощью метода remove() и вычисление количества элементов через функцию len().(Рис.5).

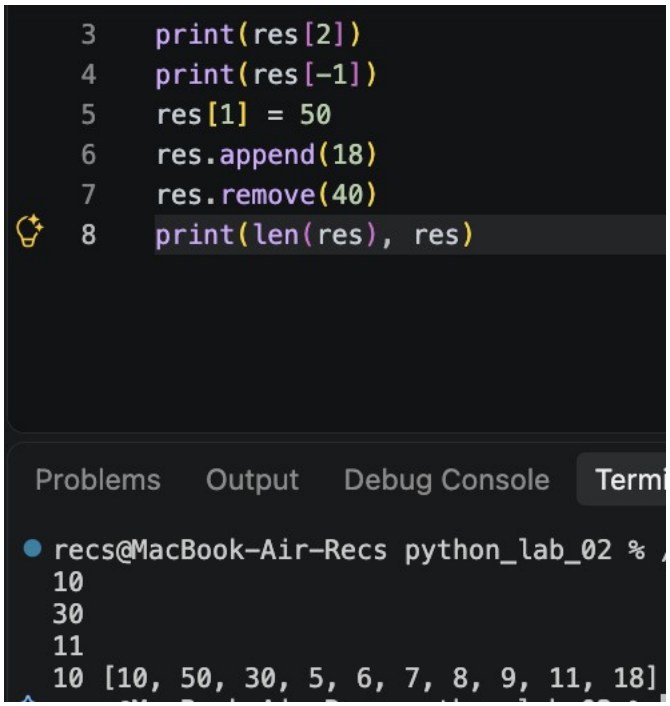


Рисунок 5 — изменение списка.

Шаг 5. Срезы.

На рисунке 6 представлены выводы программы по срезам для указанного списка.

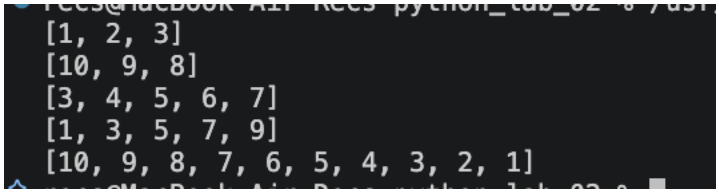


Рисунок 6 — Срезы.

Шаг 6. Работа с кортежами.

На рисунке 7 представлен вывод элементов из кортежа с помощью срезов.

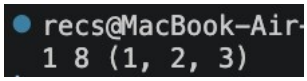


Рисунок 7 — Элементы кортежа.

При попытке изменить кортеж появляется ошибка так как кортеж является неизменяемым объектом. (рис 8).

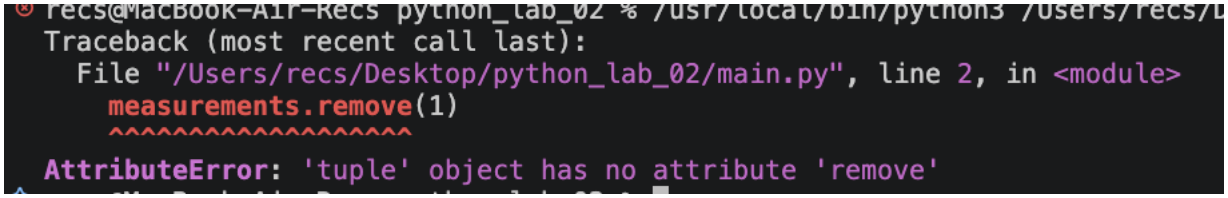


Рисунок 8 — Ошибка при изменении кортежа.

На рисунке 9 представлено выполнение операции вывода элемента из кортежа.

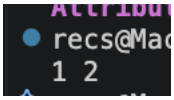


Рисунок 9 — Распаковка элемента из кортежа.

На рисунке 10 представлен выход программы после преобразования кортежа в список, изменения элемента и преобразования списка в кортеж.

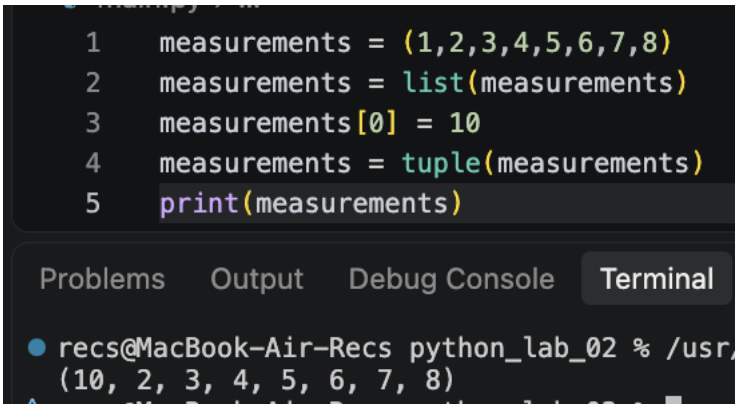


Рисунок 10 — Преобразование кортежа.

Шаг 7. Встроенные функции.

На рисунке 11 представлен вывод программы в которой используются основные функции над списком.

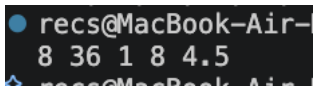


Рисунок 11 — Встроенные программы.

Функция sorted() добавляет отсортированный список в новую переменную, а метод .sort() сортирует сам список.(Рис 12)

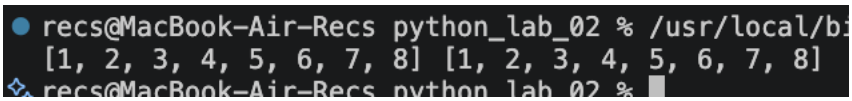


Рисунок 12 — функция sorted() и метод .sort().

Шаг 8. Реализация собственных алгоритмов.

На рисунке 13 представлена самостоятельная реализация функций sum(), min(), max().

```
1   def calculate_sum(numbers):
2       total = 0
3       for num in numbers:
4           total = total + num
5       return total
6
7   def find_min(numbers):
8       current_min = numbers[0]
9       for num in numbers:
10          if num < current_min:
11              current_min = num
12      return current_min
13
14  def find_max(numbers):
15      current_max = numbers[0]
16      for num in numbers:
17          if num > current_max:
18              current_max = num
19      return current_max
```

Рисунок 13 — Реализация функций.

На рисунке 14 представлен вывод встроенных функций и вывод собственных, выводы не отличаются значит функции работают правильно.

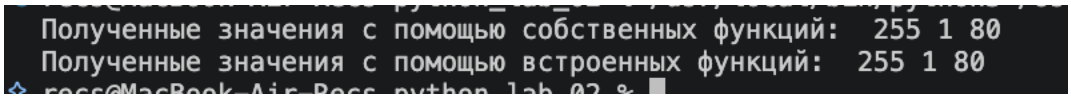


Рисунок 14 — Сравнение результатов.

Шаг 9. Линейный поиск.

На рисунке 15 представлена программа find_element и ее вывод для существующего числа и отсутствующего.

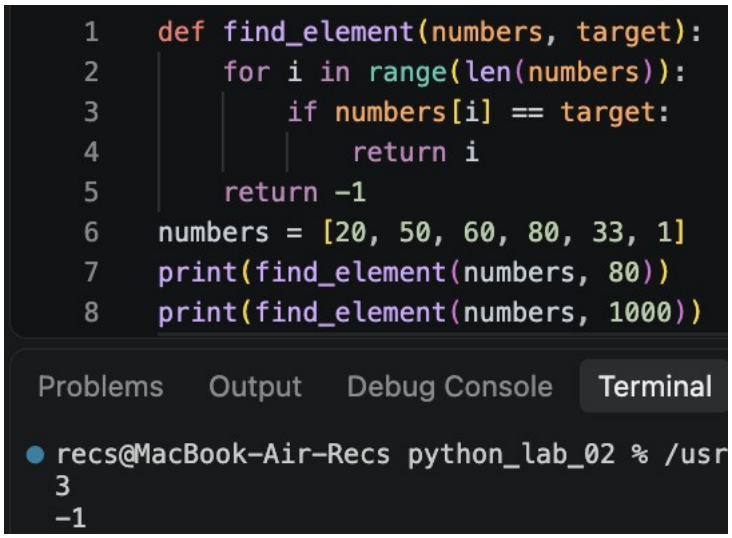


Рисунок 15 — Функция поиска.

Шаг 10. Возврат нескольких значений.

На рисунке 16 представлена функция analyze, которая возвращает кортеж из трех чисел и ее вывод в отдельные переменные.

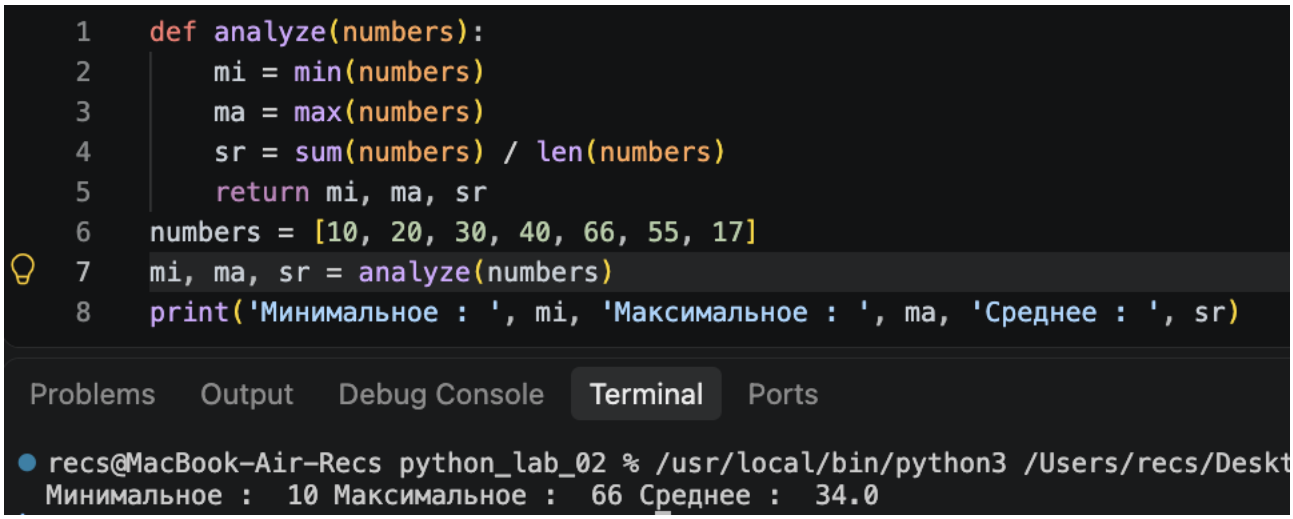


Рисунок 16 — Функция analyze.

Шаг 11. Lambda-функции.

На рисунке 17 представлен вывод обычной сортировки и сортировки по абсолютному значению элементов. Обычная сортировка упорядочивает числа по их значению, поэтому отрицательные числа оказываются в начале списка. При сортировке с параметром key и lambda-функцией lambda x: abs(x)элементы сравниваются по модулю, поэтому порядок иной. Lambda-функция задаёт критерий сравнения элементов, в данном примере — абсолютное значение числа.

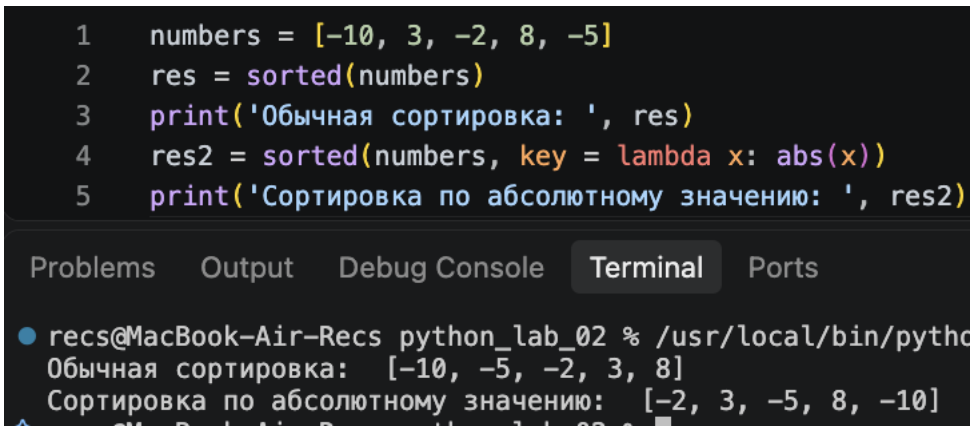


Рисунок 17 — Использование lambda-функции.

Шаг 12. Самостоятельная работа.

На рисунке 18 представлен вывод программы по анализу списка чисел.

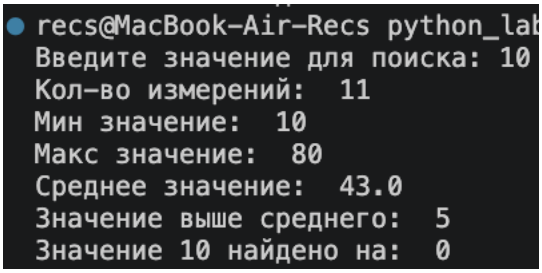


Рисунок 18 — Вывод программы.

На рисунках 19 — 20 представлен код программы.

```
def calculate_average(measurements):
    total = 0
    for value in measurements:
        total += value
    return total / len(measurements)
def find_minimum(measurements):
    current_mi = measurements[0]
    for value in measurements:
        if value < current_mi:
            current_mi = value
    return current_mi
def find_maximum(measurements):
    current_ma = measurements[0]
    for value in measurements:
        if value > current_ma:
            current_ma = value
    return current_ma
def count_above_average(measurements):
    average = calculate_average(measurements)
    count = 0
    for value in measurements:
        if value > average:
            count += 1
    return count
```

Рисунок 19 — Код программы.

```
    return count
def find_value(measurements, target):
    for i in range(len(measurements)):
        if measurements[i] == target:
            return i
    return -1
def analyze_measurements(measurements):
    return find_minimum(measurements), find_maximum(measurements), calculate_average(measurements)
measurements = [10,20,30,40,50,60,70,80,69,33,11]
target = int(input('Введите значение для поиска: '))
mi, ma, av = analyze_measurements(measurements)
print('Кол-во измерений: ', len(measurements))
print('Мин значение: ', mi)
print('Макс значение: ', ma)
print('Среднее значение: ', av)
print('Значение выше среднего: ', count_above_average(measurements))
found_index = find_value(measurements, target)
if found_index != -1:
    print('Значение', target, 'найдено на: ', found_index)
else:
    print('Значение', target, 'отсутствует')
```

Рисунок 20 — Код программы.

Вывод

В ходе выполнения лабораторной работы изучены функции в Python: определение, параметры, оператор return и аргументы по умолчанию, а также различие локальных и глобальных переменных. Освоена работа со списками и кортежами: индексация, срезы, методы списков и распаковка; подтверждено, что список изменяем, а кортеж — нет. Самостоятельно реализованы алгоритмы суммирования, поиска минимума, максимума и линейного поиска; их результаты совпали со встроенными функциями.